Task Description

```
def truncate_number(number: float) -> float:
    """ Given a positive floating point number, it can be
decomposed into and integer part (largest integer smaller
than given number) and decimals
    (leftover part always smaller than 1).

    Return the decimal part of the number.
    >> truncate_number(3.5)
    0.5
    """
```

Prompt Agent Prompt Template

```
Prompt_example ="""
### Problem:
Write a function called factorial that computes the factorial of a given non-negative integer n, where the factorial of a number is the product
of all positive integers up to that number.
### Chain of thought:
First, we recognize that the factorial of a number is the product of all positive integers from 1 to that number.
There are two common approaches to calculating the factorial: iteratively and recursively.
For this task, we'll go with the iterative approach as it's straightforward and avoids potential issues with recursion limits for larger numbers.
The iterative approach involves initializing a variable to 1 and then multiplying it with every integer from 1 to n.
We also need to handle the edge case where n is 0, since 0! (0 factorial) is defined as 1.
Finally, we'll test the function to ensure it works correctly.
"""
@ell.simple(model="gpt-3.5-turbo-0613", client=client)
def Prompt_Agent(problem: str):
    """You are a Prompt Agent."""# System prompt
    return f"Here is an example.\n"+Prompt_example+"Please write chain of thought for the problem:\n {problem} \n .Please only provide chain of
thought."# User prompt
```

Prompt Agent Output

```
To extract the decimal part of a floating-point number, we can use the `math.floor` function to obtain the integer part. By subtracting this
integer from the original number, we will get the decimal part. Since the input is a positive float, we don't need to handle negative numbers or
special cases. After implementing this logic, we can test the function to ensure accuracy.
```

Test Agent Prompt Template

```
Test_example ="""
### Problem:
Write a function called factorial that computes the factorial of a given
non-negative integer n, where the factorial of a number is the product of
all positive integers up to that number.

### Test Cases:
assert factorial(0)==1 # factorial(0) should return 1
assert factorial(0)==1 # factorial(1) should return 1
assert factorial(2)==2 # factorial(2) should return 2
assert factorial(5)==120 # factorial(5) should return 120
assert factorial(10)==3628800 # factorial(10) should return 3628800
"""
@ell.simple(model="gpt-3.5-turbo-0613", client=client)
def Test_Agent(problem: str):
    """You are a Test Agent."""  # System prompt

    return(
        "Here is an example.\n"
        +Test_example
        +"Please generate three tests for the problem:\n"
        +f"{problem}\n"
        "Please only provide the tests."
    )
```

Coding Agent Prompt Template

```
@ell.simple(model="gpt-3.5-turbo-0613", client=client)
def Code_Agent(problem: str, cot: str):
    """You are a Code Agent."""

    return(
        "Please write code for the problem:\n"
        +f"{problem}\nMore information:\n"
        +f"{cot}\nPlease only provide the code."
    )
```

Repair Agent Prompt Template

```
@ell.simple(model="gpt-3.5-turbo-0613", client=client)
def Repair_Agent(code: str, feedback: str):
    """You are a Repair Agent."""

    return(
        "Please write repair advice for the code:\n"
        +f"{code}\nHere is the feedback of compiler:\n"
        +f"{feedback}\nPlease only provide the advice."
    )
```

Test Agent Output

```
assert truncate_number(3.5) == 0.5   # 3.5 has a decimal part of 0.5
assert truncate_number(5.9) == 0.9   # 5.9 has a decimal part of 0.9
assert truncate_number(10.0) == 0.0  # 10.0 has a decimal part of 0.0
assert truncate_number(7.25) == 0.25 # 7.25 has a decimal part of 0.25
assert truncate_number(0.1) == 0.1   # 0.1 has a decimal part of 0.1
```

Repair Agent Output

```
It looks like there's an issue with the `truncate_number`
function when it comes to handling certain inputs, like `5.9`.
### Current Function Behavior
The function is designed to return the decimal part of a float.
It does ...
### Why the Test Failed
The failing test case:
    assert truncate_number(5.9) == 0.9
indicates that the function returned a value that does not
match the expected `0.9`...
### Step-by-Step Analysis
1. Check the Function Logic:
   - For `5.9`, `math.floor(5.9)` returns `5`.
   - The calculation is `5.9 - 5 = 0.9`, which should work as
intended...
### Possible Issues
Environment: Ensure that the testing environment does not
alter floating-point representation...
```

Coding Agent Output

```
def truncate_number(number: float) -> float:
    """
    >>> truncate_number(3.5)
    0.5
    >>> truncate_number(4.75)
    0.75
    """
    import math
    return number -math.floor(number)
```